

001 - Frontend Foundation and AI Agent Coding

Who this note is for

This note is for FDE candidates learning frontend from a practical product-building point of view.

The goal is not to memorize every frontend tool. The goal is to understand how a frontend application works end to end, how to build a small working app, and how to use AI coding agents responsibly while still owning the result.

By the end, you should be able to explain and build this flow:

```
User action -> frontend state -> API request -> response -> UI update
```

1. What frontend really means

Frontend is the user-facing layer of a software product.

It is not only "making screens look good." A good frontend owns the full user experience:

- what the user wants to do
- what the user sees
- what input the user gives
- what the app remembers right now
- how the app talks to APIs
- how the app responds to loading, errors, empty data, and success

Simple mental model:

```
Frontend = user intent -> interface -> state -> network -> feedback
```

Example:

```
User wants weather for Nagpur
-> enters "Nagpur"
-> clicks Search
-> app stores the input
-> app shows loading
-> app calls weather API
-> app receives JSON
-> app displays temperature, humidity, wind speed
-> if something fails, app shows an error
```

That complete loop is frontend.

2. Short history of frontend

Frontend became complex because the browser changed from a document viewer into an application runtime.

1990s: static pages

Websites were mostly linked HTML documents.

Click link -> browser loads a new page

2000s: AJAX

Pages could update parts of the UI without reloading the whole page.

Click button -> JavaScript calls API -> update page

2010s: single-page applications

Apps like Gmail made the browser behave like a full application.

One loaded app -> many screens -> state changes inside browser

2020s and beyond: full-stack frontend and AI surfaces

Modern frontend now serves:

- human users
- search engines
- AI answer engines
- crawlers
- browser agents
- mobile/webview users

This is why frontend is no longer just HTML pages or visual design.

3. The browser is the frontend runtime

Your frontend code does not run "inside React."

It runs inside the browser.

React, Vite, Next.js, Tailwind, and other tools eventually produce HTML, CSS, and JavaScript that the browser understands.

The browser does this work:

```
parse HTML
fetch CSS/JS/assets
execute JavaScript
build the DOM
calculate layout
paint pixels
listen for events
send network requests
update the page
```

Core browser pieces:

Piece	Meaning
HTML	Structure of the page
CSS	Visual styling and layout
JavaScript	Behavior and interaction
DOM	Live tree of elements on the page
Events	User/browser actions like click, input, submit
Network	API calls, files, images, backend communication

Example:

```
<button id="loadButton">Load Fact</button>
<p id="result"></p>
```

```
const button = document.querySelector("#loadButton");
const result = document.querySelector("#result");

button.addEventListener("click", () => {
  result.textContent = "Button clicked";
});
```

What happens:

```
User clicks button
-> browser fires click event
```

```
-> JavaScript handler runs  
-> DOM changes  
-> browser repaints the text
```

4. Browsers are everywhere

Do not design only for your laptop.

Frontend can run on:

- desktop browsers
- mobile browsers
- tablets
- embedded webviews
- smart displays
- internal dashboards
- public websites

Real-world frontend must consider:

- screen size
- touch vs mouse vs keyboard
- weak network
- slow devices
- accessibility
- readable text
- loading feedback
- error recovery

Example:

A CRM dashboard may be desktop-first, but a lead capture form should still work on mobile.

5. Six basics before React

Before learning React deeply, understand these six basics.

1. HTML

HTML defines what exists on the page.

Use meaningful structure:

```
<main>  
  <h1>City Weather</h1>  
  <form>  
    <label for="city">City</label>  
    <input id="city" name="city" />
```

```
<button type="submit">Search</button>
</form>
</main>
```

Good HTML helps:

- users
- screen readers
- search engines
- maintainers
- testing tools

2. CSS

CSS controls layout, spacing, colors, typography, and responsiveness.

Basic example:

```
.container {
  max-width: 720px;
  margin: 0 auto;
  padding: 24px;
}

.result-card {
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 16px;
}
```

Good CSS makes the UI readable and stable.

3. JavaScript

JavaScript handles interaction and logic.

```
const city = cityInput.value.trim();

if (!city) {
  showError("Please enter a city name.");
  return;
}
```

JavaScript is used for:

- reading input
- validating forms
- calling APIs

- updating UI
- handling async behavior

4. HTTP and APIs

Frontend often talks to an API.

Basic API request:

```
const response = await fetch("https://api.example.com/data");
const data = await response.json();
```

Important API concepts:

- URL
- method: GET, POST, PUT, DELETE
- headers
- request body
- response body
- status codes
- JSON

5. DOM and state

The DOM is what is visible on the page.

State is what the app remembers.

Example state:

```
let state = {
  query: "",
  loading: false,
  error: "",
  result: null,
};
```

Frontend is mostly about changing state and rendering the correct UI for that state.

6. Accessibility

Accessibility means the app is usable by real people in real conditions.

Basic habits:

- use labels for inputs
- make buttons actual `<button>` elements
- keep text readable

- support keyboard navigation
 - do not rely only on color
 - provide alt text for meaningful images
 - show clear errors
-

6. The frontend state loop

Every useful frontend screen has states.

For an API-based search app, common states are:

```
idle
loading
success
error
empty
```

Example:

```
const state = {
  loading: false,
  error: '',
  result: null,
};
```

When user clicks Search:

```
validate input
-> clear old error
-> set loading true
-> call API
-> if success, store result
-> if error, store error message
-> set loading false
-> render UI
```

Simple example:

```
async function searchWeather(city) {
  setLoading(true);
  clearError();
  clearResult();

  try {
    const data = await fetchWeather(city);
```

```
    showResult(data);  
  } catch (error) {  
    showError("Could not fetch weather. Please try again.");  
  } finally {  
    setLoading(false);  
  }  
}
```

The user should never be confused about what is happening.

Bad:

User clicks Search and nothing visibly changes.

Good:

User clicks Search -> button disables -> "Loading..." appears -> result or error appears.

7. API calling from frontend

For the mini project, you will use API Ninjas.

API Ninjas usually expects this header:

X-Api-Key: YOUR_API_KEY

Learning-only example:

```
const API_KEY = "YOUR_API_KEY";  
  
async function fetchWeather(city) {  
  const url = `https://api.api-ninjas.com/v1/weather?  
city=${encodeURIComponent(city)}`;  
  
  const response = await fetch(url, {  
    headers: {  
      "X-Api-Key": API_KEY,  
    },  
  });  
  
  if (!response.ok) {  
    throw new Error(`API request failed with status ${response.status}`);  
  }  
}
```

```
    return response.json();  
  }
```

Important security note:

For this learning exercise, using the API key in frontend code is acceptable.

In production, do not expose secret API keys in frontend code. Use a backend or server-side proxy.

8. Example mini project: City Weather Info

User flow

```
User opens app  
-> sees title and city input  
-> enters Nagpur  
-> clicks Search  
-> app validates input  
-> app shows loading  
-> app calls API Ninjas Weather API  
-> API returns data  
-> app displays weather  
-> if error happens, app shows error
```

Minimal HTML

```
<main class="container">  
  <h1>City Weather Info</h1>  
  
  <form id="weatherForm">  
    <label for="cityInput">City name</label>  
    <input id="cityInput" type="text" placeholder="Example: Nagpur" />  
    <button id="searchButton" type="submit">Search</button>  
  </form>  
  
  <p id="message"></p>  
  <section id="result"></section>  
</main>
```

Minimal JavaScript

```
const API_KEY = "YOUR_API_KEY";  
  
const form = document.querySelector("#weatherForm");  
const cityInput = document.querySelector("#cityInput");
```

```
const message = document.querySelector("#message");
const result = document.querySelector("#result");
const searchButton = document.querySelector("#searchButton");

form.addEventListener("submit", async (event) => {
  event.preventDefault();

  const city = cityInput.value.trim();

  if (!city) {
    showMessage("Please enter a city name.", "error");
    result.innerHTML = "";
    return;
  }

  setLoading(true);
  showMessage("Loading weather...", "info");
  result.innerHTML = "";

  try {
    const data = await fetchWeather(city);
    renderWeather(data, city);
    showMessage("", "info");
  } catch (error) {
    showMessage("Could not fetch weather. Please try again.", "error");
  } finally {
    setLoading(false);
  }
});

async function fetchWeather(city) {
  const url = `https://api.api-ninjas.com/v1/weather?city=${encodeURIComponent(city)}`;

  const response = await fetch(url, {
    headers: {
      "X-API-Key": API_KEY,
    },
  });

  if (!response.ok) {
    throw new Error(`Request failed: ${response.status}`);
  }

  return response.json();
}

function renderWeather(data, city) {
  result.innerHTML = `
    <article class="result-card">
      <h2>${city}</h2>
      <p>Temperature: ${data.temp ?? "N/A"} C</p>
      <p>Humidity: ${data.humidity ?? "N/A"}%</p>
      <p>Wind speed: ${data.wind_speed ?? "N/A"}</p>
    </article>
  `;
}
```

```
    </article>
  `;
}

function setLoading(isLoading) {
  searchButton.disabled = isLoading;
  searchButton.textContent = isLoading ? "Searching..." : "Search";
}

function showMessage(text, type) {
  message.textContent = text;
  message.className = type;
}
```

What this example demonstrates

- reading user input
- validating empty input
- handling form submit
- calling an external API
- using an API key header
- showing loading state
- showing error state
- rendering result data
- avoiding crashes when fields are missing

9. Folder structure

Use a structure that matches your skill level.

Simple HTML/CSS/JS

```
frontend-mini-project/
  index.html
  styles.css
  script.js
  README.md
```

Good for beginners.

Slightly organized JavaScript

```
frontend-mini-project/
  index.html
  src/
    api.js
    main.js
```

```
ui.js
styles/
  styles.css
README.md
```

Good when you want to separate API logic from UI logic.

React structure

```
frontend-mini-project/
  src/
    app/
      App.jsx
    features/
      search/
        SearchForm.jsx
        ResultCard.jsx
        api.js
    shared/
      components/
        Loader.jsx
        ErrorMessage.jsx
    main.jsx
  README.md
```

Good when using React.

Rule:

Keep related UI, API calls, types, hooks, and tests close to the feature that changes.

For example, in a CRM:

```
features/
  contacts/
    ContactTable.jsx
    ContactForm.jsx
    api.js
    types.js
```

If contact requirements change, you know where to look.

10. Stack selection

Do not choose a stack because it is popular.

Choose based on product needs.

Product need	Possible stack
Simple static page	HTML/CSS/JS
Small demo app	HTML/CSS/JS or React + Vite
Internal dashboard	React + Vite + TypeScript
SEO-heavy public site	Next.js or Astro
Forms-heavy app	React Hook Form + Zod
Server API data with caching	TanStack Query
Accessible component system	Radix UI or shadcn/ui

Recommended beginner frontend path:

```
HTML -> CSS -> JavaScript -> APIs -> React -> TypeScript -> Vite ->
Tailwind -> forms -> server state
```

11. React mental model

React is a library for building UI from components and state.

Basic idea:

```
state changes -> component re-renders -> UI updates
```

Simple React example:

```
import { useState } from "react";

export function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increase</button>
    </div>
  );
}
```

React does not remove the browser fundamentals.

React still depends on:

- HTML concepts
- CSS layout
- JavaScript
- events
- state
- API calls
- accessibility

If you understand the browser flow, React becomes easier.

12. API and async state in React

Basic React API example:

```
import { useState } from "react";

const API_KEY = "YOUR_API_KEY";

export function WeatherSearch() {
  const [city, setCity] = useState("");
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");
  const [weather, setWeather] = useState(null);

  async function handleSearch(event) {
    event.preventDefault();

    if (!city.trim()) {
      setError("Please enter a city name.");
      setWeather(null);
      return;
    }

    setLoading(true);
    setError("");
    setWeather(null);

    try {
      const response = await fetch(
        `https://api.api-ninjas.com/v1/weather?city=${encodeURIComponent(city)}`,
        {
          headers: {
            "X-API-Key": API_KEY,
          },
        }
      );
    }
  }
}
```

```

    if (!response.ok) {
      throw new Error("Request failed");
    }

    const data = await response.json();
    setWeather(data);
  } catch (error) {
    setError("Could not fetch weather. Please try again.");
  } finally {
    setLoading(false);
  }
}

return (
  <main>
    <h1>City Weather Info</h1>

    <form onSubmit={handleSearch}>
      <label htmlFor="city">City</label>
      <input
        id="city"
        value={city}
        onChange={(event) => setCity(event.target.value)}
      />
      <button disabled={loading}>{loading ? "Searching..." : "Search"}
    </button>
    </form>

    {error && <p role="alert">{error}</p>}

    {weather && (
      <section>
        <p>Temperature: {weather.temp ?? "N/A"} C</p>
        <p>Humidity: {weather.humidity ?? "N/A"}%</p>
        <p>Wind speed: {weather.wind_speed ?? "N/A"}</p>
      </section>
    )}
  </main>
);
}

```

What to notice:

- input value is state
- loading is state
- error is state
- API response is state
- UI is based on current state

13. Media in frontend

Media can make products useful and rich, but it can also destroy performance.

Images

Good habits:

- use correct dimensions
- compress large images
- lazy-load below-the-fold images
- use alt text for meaningful images
- avoid loading huge originals into tiny cards

Example:

```

```

Video

Think about:

- file size
- streaming
- poster image
- captions
- mobile bandwidth

Audio

Think about:

- browser permissions
- microphone selection
- latency
- user feedback
- recording state

File uploads

A good upload UI should show:

- accepted file types
- file size limits
- preview
- progress
- success state

- error state

14. SEO and AI search visibility

SEO matters for public pages, not private dashboards.

Examples where SEO matters:

- landing page
- blog
- product page
- documentation
- pricing page
- public help center

Examples where SEO usually does not matter:

- private CRM dashboard
- admin panel
- authenticated internal tool

Search engines need to:

```
crawl -> render -> index -> rank/surface
```

Frontend basics for SEO:

- meaningful page title
- useful meta description
- clear headings
- readable text content
- clean URLs
- internal links
- fast loading
- accessible HTML
- structured data when useful

AI search systems also need clear, factual, crawlable content. The same basics help:

- direct answers
- structured headings
- visible text
- trustworthy sources
- useful page organization

Bad public page:

Important content hidden only inside an image.

Good public page:

Important content available as structured text with clear headings.

15. Coding with AI agents

AI coding agents can help you move faster, but they do not remove your responsibility.

You are still the engineer.

The agent can:

- inspect files
- explain code
- generate components
- suggest folder structure
- write API call logic
- debug errors
- run tests
- improve README files
- review code

You must:

- give clear instructions
- review the generated code
- understand what changed
- test the app
- protect secrets
- avoid blindly accepting output
- explain the final solution yourself

16. Good AI-agent workflow for frontend

Use this process when building your mini project.

Step 1: Explain the goal

Good prompt:

I need to build a beginner frontend-only app using HTML, CSS, and JavaScript.

It should use API Ninjas Weather API.
The app should let the user enter a city, show loading state, show error state,
and display temperature, humidity, and wind speed.
Create a simple folder structure and explain each file.

Bad prompt:

Make frontend app.

Step 2: Ask for a small plan first

Before writing code, give me a short implementation plan:
files, UI sections, state variables, API function, and error handling.

Step 3: Build in small pieces

Ask for one layer at a time:

Create only the HTML structure first.

Now add CSS for a clean responsive layout.

Now add JavaScript for form submit, validation, loading, API call, and rendering.

Step 4: Ask the agent to explain the flow

Explain this app using:
User action -> frontend state -> API request -> response -> UI update.

Step 5: Ask for review

Review this code for beginner mistakes:
missing error handling, exposed assumptions, bad variable names,
broken loading state, and UI issues.

Step 6: Verify manually

Do not stop at "agent says it works."

You should manually test:

- empty input
- valid input
- wrong input
- network failure
- missing API key
- mobile screen width
- repeated button clicks

17. What to ask an AI agent while learning

Use AI agents as a tutor, not only a code generator.

Useful questions:

Explain what happens when I click this button.

Where is the API call happening in this code?

What state variables does this app have?

What can go wrong in this API request?

How should I show loading, error, empty, and success states?

Can you simplify this code for a beginner?

Can you split this code into api.js and ui.js?

Can you write a README that explains how to run the app?

Can you quiz me on this code so I know whether I understand it?

18. Mistakes to avoid with AI-generated frontend

Avoid these common mistakes:

- copying code without understanding it
- using too many libraries for a small task
- ignoring loading state
- ignoring error state
- showing only `console.log(error)` instead of UI feedback
- exposing real production secrets
- accepting messy folder structure
- building features not required by the task
- using React before understanding the basic flow
- making a beautiful UI that does not actually work
- not testing empty input
- not checking mobile layout
- not writing a README

Remember:

A small working app you can explain is better than a complex app you cannot understand.

19. Mini project options

Choose one.

Option 1: Facts Generator

User clicks a button and receives a random fact.

Good for:

- beginners
- button event handling
- loading and error state

Core flow:

```
Click button -> loading -> Facts API -> show fact
```

Option 2: City Weather Info

User enters a city and receives weather information.

Good for:

- forms
- validation
- API query parameters
- structured result display

Core flow:

```
Enter city -> validate -> Weather API -> show weather
```

Option 3: Domain Lookup Tool

User enters a domain and receives domain information.

Good for:

- search-style interfaces
- structured technical data

Core flow:

```
Enter domain -> validate -> Domain API -> show domain info
```

Option 4: Nutrition Search App

User enters food and receives nutrition information.

Good for:

- displaying lists or structured fields
- handling missing fields

Core flow:

```
Enter food -> validate -> Nutrition API -> show calories/macros
```

Option 5: Quotes App

User clicks a button and receives a quote.

Good for:

- simple polished UI
- button-based API call

Core flow:

Click button -> loading -> Quotes API -> show quote and author

20. Minimum project requirements

Your project must include:

UI requirements

- clear title
- at least one input or button
- result display section
- loading indicator
- error message area
- basic responsive layout

Functional requirements

- call at least one API Ninjas endpoint
- use the API key correctly
- display real API response data
- handle empty input if your app uses input
- handle API failure gracefully
- avoid breaking the UI if response fields are missing

Code requirements

- readable code
- meaningful variable names
- API logic separated if possible
- minimal copy-paste
- basic comments only where useful

21. Demo preparation

Be ready to answer:

1. What did you build?
2. Which API did you use?

3. Where is the API call happening?
4. How are you handling loading state?
5. How are you handling errors?
6. What happens if the user gives empty input?
7. How did you structure your files?
8. What would you improve with more time?

Your demo should be short and clear:

```
I built a City Weather app.  
The user enters a city and clicks Search.  
The app validates the input, sets loading state, calls API Ninjas Weather  
API,  
then displays temperature, humidity, and wind speed.  
If input is empty or the API fails, the app shows an error message.
```

22. README template

Include a **README.md** in your project.

Example:

```
# City Weather Info

## What it does

This is a frontend-only weather app. The user enters a city name and the  
app  
fetches weather data from API Ninjas.

## API used

API Ninjas Weather API.

## How to run

Open `index.html` in the browser.

Or, if using a local server:

```bash
npx serve .
```

## Features

- City input
- Empty input validation
- API request using X-API-Key header
```

- Loading state
- Error state
- Weather result card
- Responsive layout

What I learned

I learned how frontend captures user input, stores UI state, calls an external API, handles loading and error states, and updates the page with response data.

23. Self-review checklist

Before submitting, check:

- ☐ Does the app run?
- ☐ Is the API key added correctly for the learning task?
- ☐ Does valid input show real data?
- ☐ Does empty input show a useful message?
- ☐ Does failed API request show an error?
- ☐ Does the UI show loading while waiting?
- ☐ Does the UI avoid crashing if a field is missing?
- ☐ Is the layout readable on mobile?
- ☐ Are file names clear?
- ☐ Is the README complete?
- ☐ Can you explain the app without reading the code line by line?

24. Final mental model

If you remember only one thing, remember this:

Frontend turns user intent into product behavior.

For every screen you build, ask:

What does the user want?
What input do they give?
What state does the app need?
What API or data source is involved?
What feedback should the user see?
What can go wrong?
How will the UI recover?

That is the foundation of practical frontend engineering.